

# Day 57 -- Line-by-Line Syntax Explanation

day57\_rnn\_forward\_pass.py, segment by segment | MD Mutasim Billah

---

## 1. rnn\_step\_forward/rnn\_forward -- the Recurrence, Made Literal

one timestep, then a loop that reuses it

```
def rnn_step_forward(x_t, h_prev, Wxh, Whh, bh):
    return np.tanh(x_t @ Wxh + h_prev @ Whh + bh) # the ENTIRE recurrence, one line

def rnn_forward(X, h0, Wxh, Whh, bh):
    ...
    for t in range(T): # walk through the sequence ONE STEP AT A TIME
        x_t = X[:, t:t + 1] # this timestep's input, kept 2D as (batch, 1)
        h = rnn_step_forward(x_t, h, Wxh, Whh, bh) # the SAME weights, reused at every t
        H[:, t, :] = h # store this timestep's hidden state
```

rnn\_step\_forward is called T times inside the loop, but the SAME Wxh, Whh, bh objects are passed in every single call -- nothing about the parameters changes with t. This is the literal code expression of "parameter sharing across time": one small function, applied repeatedly.

## 2. VanillaRNNClassifier -- No Sequence-Length Argument

the constructor that doesn't need to know T

```
def __init__(self, hidden_dim=8, n_classes=2, seed=42): # NOTE: no T argument anywhere
    self.Wxh = rng.randn(1, hidden_dim) * he_scale(1) # shape depends on hidden_dim, NOT T
    self.Whh = rng.randn(hidden_dim, hidden_dim) * he_scale(hidden_dim) # shape depends on
    hidden_dim, NOT T
```

Compare this to Day 56's PlainFCNet.\_\_init\_\_(self, T, hidden=16, ...), whose very first argument is T and whose W1 shape (T, hidden) is BUILT from it. Here, every weight's shape depends only on hidden\_dim and the (fixed, small) input width per timestep -- never on how long the sequence is, which is exactly why any T can be handled by the same network.

## 3. demo\_correctness\_check() -- Verifying Without Gradients

hand-computing h1, h2, h3 explicitly, one call at a time

```
h1_hand = np.tanh(X[:, 0:1] @ Wxh + h0 @ Whh + bh) # h1, using h0
h2_hand = np.tanh(X[:, 1:2] @ Wxh + h1_hand @ Whh + bh) # h2, using h1_hand -- NOT H_loop
h3_hand = np.tanh(X[:, 2:3] @ Wxh + h2_hand @ Whh + bh) # h3, using h2_hand
```

Each hand-computed line feeds the PREVIOUS hand-computed result forward (h1\_hand into h2\_hand's formula), exactly mirroring the recurrence -- deliberately never reading from H\_loop (the thing being checked), so the comparison is genuinely independent rather than circular.

## 4. demo\_long\_range\_sensitivity() -- Finite Differences on x\_0

### **perturbing only the FIRST timestep, reading the LAST hidden state**

```
X_plus = X.copy(); X_plus[0, 0] += eps # nudge ONLY x_0 up
X_minus = X.copy(); X_minus[0, 0] -= eps # nudge ONLY x_0 down
h_T_plus = rnn_forward(X_plus, h0, ...)[:, -1, :] # the FINAL hidden state after the up-nudge
h_T_minus = rnn_forward(X_minus, h0, ...)[:, -1, :] # the FINAL hidden state after the
down-nudge
sensitivity = np.linalg.norm(h_T_plus - h_T_minus) / (2 * eps) # central-difference estimate
```

This is the same central-difference technique every prior day's gradient checks used -- applied here to a quantity that isn't a loss gradient at all, just "how much does the output change when this one input changes," a general-purpose sensitivity measurement that works even with no backward pass defined yet.